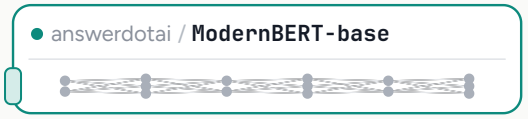
SECTION 4 CHEAT SHEET

An encoder, a random head and an honest reward

Mini Laya puts a new decision head on ModernBERT-base and trains it with Laya's own `finetune` on Kev's labelled questions. It beats Laya on the tasks it trained on; on tasks it never saw, Laya still wins.



pretrained encoder

decision head 2 layers, random

encoder	149,014,272
head	14,969,347
total	163,983,619

1 THE PARTS

Pretrained encoder plus a new decision head

The config names the encoder and the head. `build_model` joins them, and `save_checkpoint` writes an untrained Laya checkpoint that `finetune` starts from.

```
ENC = "answerdotai/ModernBERT-base"
CONFIG = {
    "encoder": ENC,
    "head_layers": 2,
    ...
}
torch.manual_seed(0)
model = build_model(CONFIG)
save_checkpoint(model, tok, CONFIG,
                "runs/mini-laya-untrained")
```

COURSE RUN untrained: choice **0.178**, Noul **0.594**, score off by **1.12**.

Kev's 123 eval files, one row per question

all rows **106,802**

the first training row

`"state"`	Can I pick a new PIN?
`"questions"`	{intent: a choice question}
`"expected"`	`{intent: change_pin}`
`"tags"`	[banking77]

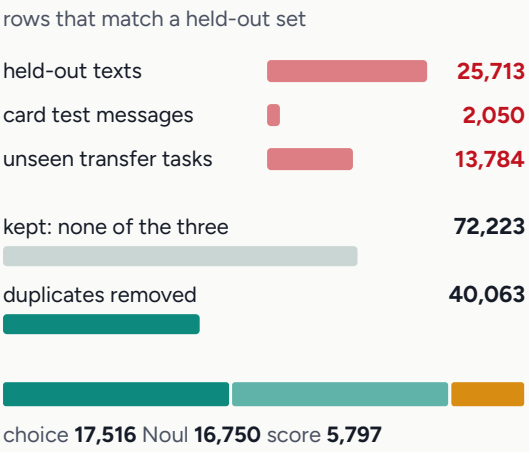
2 THE DATA

Kev's questions, one row each

Every Kev record becomes one row per question, with the gold label copied to `expected` and the task to `tags`, the same row format finetune and evaluate read.

```
for qid, question in record["questions"].items():
    rows.append({
        "state": record["state"],
        "questions": {qid: question},
        "expected": {qid: question["label"]},
        "tags": [question["src"]],
        ...
    })
kev = pd.DataFrame(rows)
```

COURSE RUN **106,802** rows from **123** files.



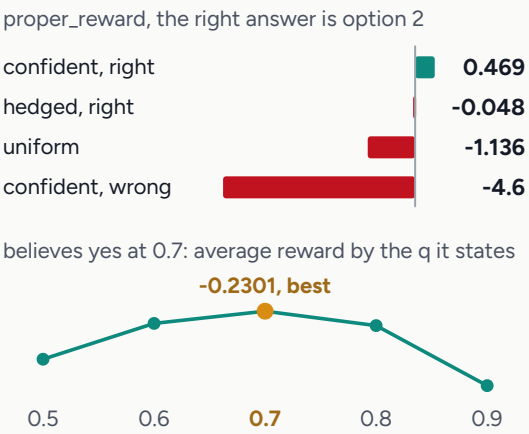
3 KEEP THE TESTS OUT

Nothing you score on ever trains

Three filters drop a row if its text sits in a held-out set or the card test, or its task is one of the unseen transfer tasks. Then the same question on the same text is kept once.

```
held_texts = held["text"]
card_texts = card_test["text"]
is_held = kev["text"].isin(held_texts)
is_card_test = kev["text"].isin(card_texts)
is_transfer_task = kev["task"].isin(transfer_tasks)
train = kev[~is_held & ~is_card_test & ~is_transfer_task]
train = train.drop_duplicates(["text", "qid"])
```

COURSE RUN held out **960** calibration, **1,592** test, **764** transfer; **373** tasks train.



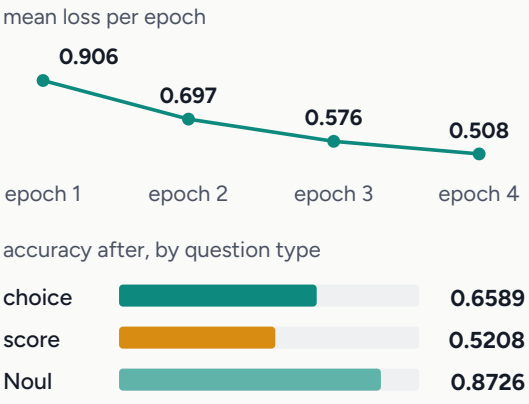
4 LAYA'S LOSS

A proper scoring rule pays for honesty

The reward is the log score plus the spherical score, minus the ranked probability score for Score questions. `rlcd`, `finetune`'s default loss, pushes toward a higher reward.

```
target = torch.tensor([[0., 1., 0., 0.]])
qtype = torch.tensor([QTYPES["choice"]])
mask = torch.ones(1, 4)
reward = proper_reward(torch.tensor([probs]), target,
                       qtype, mask)
```

COURSE RUN stating the belief itself, 0.7, earns most.



5 TRAIN

The same finetune, from the untrained checkpoint

`shuffle_options` reorders choice options every epoch, so positions teach nothing. One temperature per question type.

```
config = TrainConfig(shuffle_options=("choice",),
    eval_data="runs/system-one-data/test.jsonl")
run = finetune("runs/system-one-data/train.jsonl",
               "runs/mini-laya-untrained", OUT, config)
pd.DataFrame.from_dict(after["by_type"], orient="index")
```

COURSE RUN **38,584** trained, **2,412** updates; accuracy **0.311** to **0.698**, ECE **0.044** to **0.034**.



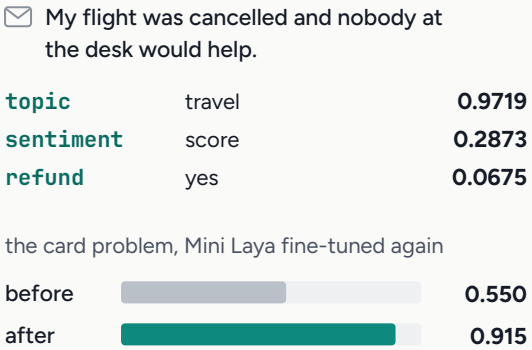
6 COMPARE

Mini Laya against Laya, same rows

`evaluate` on the same test and transfer rows, each with its own temperatures.

```
original = laya.Agent("convaiinnovations/laya",
                      device="cuda")
for model_name, agent in AGENTS.items():
    for set_name, data in SETS.items():
        report = evaluate(agent, data)
```

COURSE RUN on test, score off by **0.54** against **0.97** levels, ECE **0.030** against **0.162**.



7 USE IT

Ask it anything, fine-tune it again

It loads like Laya and answers typed questions; `finetune` it on Section 3's card rows.

```
ours = laya.Agent(OUT, device="cuda")
result = ours.system_one(..., QUESTIONS)
lab_config = TrainConfig(eval_data="runs/cards/test.jsonl")
cards = finetune("runs/cards/train.jsonl", OUT,
                 "runs/mini-laya-cards", lab_config)
```

COURSE RUN cards ECE **0.122** to **0.023**; Section 3's Laya went **0.675** to **0.9225**.

TERMS

decision head: the head that answers a typed question with calibrated probabilities.

proper scoring rule: a loss that rewards honest confidence.

checkpoint: the saved files of a trained model.

Mini Laya: the model assembled here; "from scratch" means assembling, not pretraining.

WHAT EACH CALL HANDS BACK

`build_model(CONFIG)`
encoder plus random decision head

`save_checkpoint(...)`
encoder, model.safetensors, rl_agent_config.json, tokenizer

`proper_reward(probs, target, ...)`
one reward, higher is better

`ours.system_one(state, QUESTIONS)`
an answer per question

WATCH OUT

- Laya still wins on unseen tasks: choice **0.520** against **0.579**, Noul **0.650** against **0.771**.
- Kev's sets hold Banking77 training messages: the Lab's first **0.550** is not zero-shot.
- Score questions are the weak spot: **0.521**, ECE **0.136**.
- Fix the seed: some random heads stay on a plateau far longer.